

National University of Computer and Emerging Sciences
Islamabad Campus

Compiler Construction (CS4031)

Sessional-II Exam

Course Instructor(s):

Dr. Mehreen Alam, Ms. Nirmal Tariq, Ms. Aminah Ali

Section(s): A,B,C,D,E,F,G

Total Time (Hrs): 1

Total Marks: 60

Total Questions: 6

Date: Apr 16, 2026

Roll No

Course Section

Student Signature

Do not write below this line.

Attempt all the questions.

Q. No	1	2	3	4	5	6	Total
Marks Obt							
Max Marks	10	20	10	8	4.5	7.5	60
CLO	3	3	1	1	2	2	

Instructions:

1. Show complete working where required to get full credit.
2. Attempt all questions on question paper. Any answer written elsewhere than provided space cannot be claimed for rechecking.
3. Calculator and other resource sharing is not allowed

[CLO 3: Design and implement a simple lexical analyzer and Parser for a given CFG.]

Question 1:

[10 marks]

Consider the given LL(1) parsing table with error recovery, perform stack-based LL(1) parsing for the input string: num * (+ num)

M[N, T]	(	number	)	+	-	*	\$
exp	exp → term exp'	exp → term exp'	pop	scan	scan	scan	pop
exp'	scan	scan	exp' → ε	exp' → addop term exp'	exp' → addop term exp'	scan	exp' → ε
addop	pop	pop	scan	addop → +	addop → -	scan	pop
term	term → factor term'	term → factor term'	pop	pop	pop	scan	pop
term'	scan	scan	term' → ε	term' → ε	term' → ε	term' → mulop factor term'	term' → ε
mulop	pop	pop	scan	scan	scan	mulop → *	pop
factor	factor → (exp)	factor → number	pop	pop	pop	pop	pop

National University of Computer and Emerging Sciences

Islamabad Campus

Partially filled parsing in the table is provided, complete the remaining steps of parsing process, continuing the trace until the input is either accepted or an unrecoverable error occurs.

Clearly indicate all the actions in table along with any error recovery steps (scan/pop).

Step	Stack	Input	Action
1	\$ exp	num * (+ num) \$	exp $\rightarrow$ term exp'
2	\$ exp' term	num * (+ num) \$	term $\rightarrow$ factor term'
3	\$ exp' term' factor	num * (+ num) \$	factor $\rightarrow$ number
4	\$ exp' term' number	num * (+ num) \$	match number
5	\$ exp' term'	num * (+ num) \$	term' $\rightarrow$ mulop factor term'
6	\$ exp' term' factor mulop	* (+ num) \$	mulop $\rightarrow$ *
7	\$ exp' term' factor *	* (+ num) \$	match *
8	\$ exp' term' factor	(+ num) \$	factor $\rightarrow$ (exp)
9	\$ exp' term') exp (	(+ num) \$	match (
10	\$ exp' term') exp	+ num) \$	ERROR: scan (+ removed)
11	\$ exp' term') exp	num) \$	exp $\rightarrow$ term exp'
12	\$ exp' term') exp' term	num) \$	term $\rightarrow$ factor term'
13	\$ exp' term') exp' term' factor	num) \$	factor $\rightarrow$ number
14	\$ exp' term') exp' term' number	num) \$	match number
15	\$ exp' term') exp' term'	) \$	term' $\rightarrow$ ϵ
16	\$ exp' term') exp'	) \$	exp' $\rightarrow$ ϵ
17	\$ exp' term')	) \$	match)
18	\$ exp' term'	\$	term' $\rightarrow$ ϵ
19	\$ exp'	\$	exp' $\rightarrow$ ϵ
20	\$	\$	Accept

[CLO 3: Design and implement a simple lexical analyzer and Parser for a given CFG.]

Question 2:

[10+5+5=20 marks]

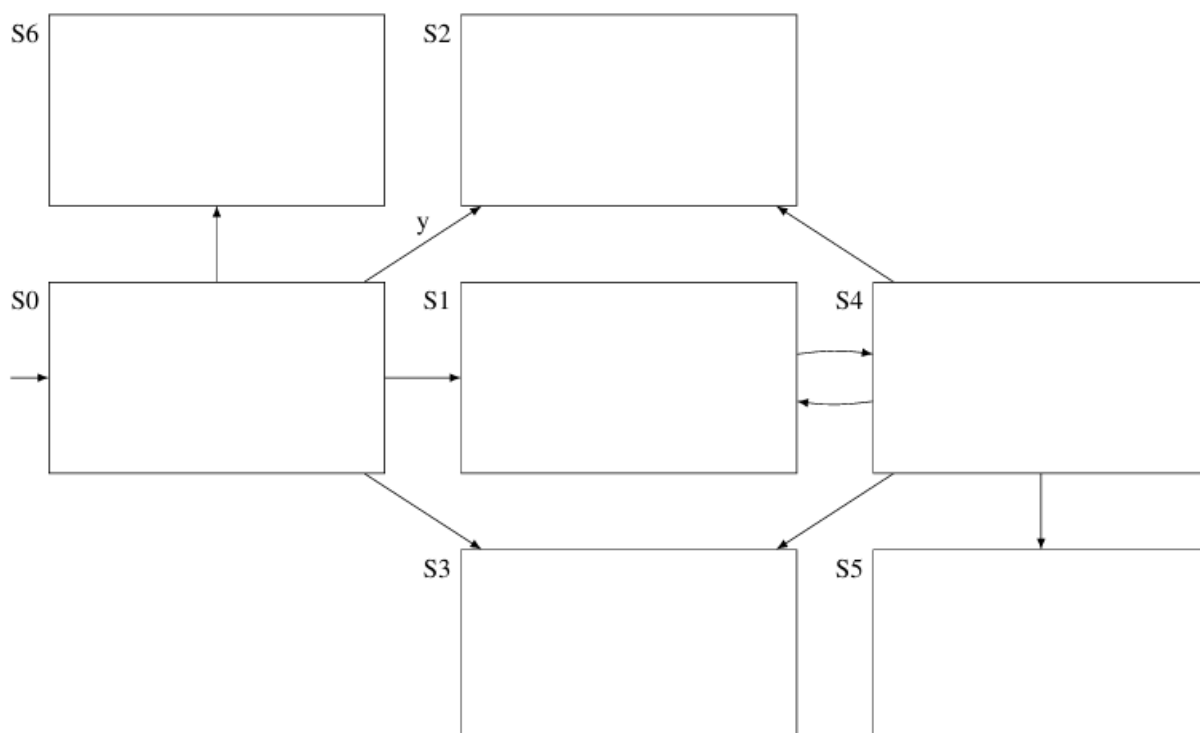
For the following grammar with terminals x, y, and z and non-terminals A', A, and B.

- 0 $A' \rightarrow A$
- 1 $A \rightarrow B \ x \ A$
- 2 $A \rightarrow y$
- 3 $B \rightarrow \epsilon$
- 4 $B \rightarrow z$

National University of Computer and Emerging Sciences

Islamabad Campus

a) Complete its LR(0) automaton. Fill in the items, indicate the accepting states, and label the unmarked transitions. No additional transitions are necessary.



b) Compute Follow sets:

Non-Terminal	Follow Sets
A'	{ \$ }
A	{ \$ }
B	{ x }

c) Fill in the SLR(1) Parse Table.

State	Action				Goto	
	x	y	z	\$	A	B
S0	$R(B \rightarrow \epsilon)$	S2	S3		6	1
S1	S4					
S2				$R(A \rightarrow y)$		
S3	$R(B \rightarrow z)$					
S4	$R(B \rightarrow \epsilon)$	S2	S3		5	1
S5				$R(A \rightarrow BxA)$		
S6				Accept		

National University of Computer and Emerging Sciences

Islamabad Campus

[CLO 1: Describe the basic phases of Compilation and recognize the theories and techniques used in each phase.]

Question 3:

[10 marks]

Classify the following grammars as LL(1), LR(0), SLR(1), LALR(1), and LR(1).

Fill the table accordingly with ✓ or X .

Grammar	LL (1)	LR (0)	SLR (1)	LALR (1)	LR (1)
$S \rightarrow aA \mid bB$ $A \rightarrow c$ $B \rightarrow c$	✓	✓	✓	✓	✓
$S \rightarrow Aa \mid bAc \mid dc \mid bda$ $A \rightarrow d$	X	X	X	✓	✓
$S \rightarrow (S)S \mid \epsilon$	✓	X	✓	✓	✓
$S \rightarrow Aa \mid bAc \mid Bc \mid bBa$ $A \rightarrow d$ $B \rightarrow d$	X	X	X	X	✓

[CLO 1: Describe the basic phases of Compilation and recognize the theories and techniques used in each phase.]

Question 4:

[8 marks]

Select the correct answer and fill in the given table below:

Sr. No.	Options			
1	A	B	C	D
2	A	B	C	D
3	A	B	C	D
4	A	B	C	D
5	A	B	C	D
6	A	B	C	D
7	A	B	C	D
8	A	B	C	D

1. An attribute grammar extends a context-free grammar by associating semantic rules with each production. In the rule $exp_1 \rightarrow exp_2 + term$, the semantic rule is: $exp_1.val = exp_2.val + term.val$. Here, $exp_1.val$ is computed entirely from values of nodes in its own subtree (its children). Based on this, what type of attribute is $exp_1.val$, and what traversal strategy correctly evaluates a grammar that uses ONLY this type of attribute?

- A) Inherited attribute; single preorder (top-down) traversal
- B) Synthesized attribute; single postorder (bottom-up) traversal
- C) Dynamic attribute; multiple passes over the parse tree
- D) Inherited attribute; multiple passes over the parse tree

National University of Computer and Emerging Sciences

Islamabad Campus

2. Consider the grammar for C-like declarations: $\text{decl} \rightarrow \text{type var-list}$ $\text{type} \rightarrow \text{int} \mid \text{float}$ $\text{var-list} \rightarrow \text{id}, \text{var-list} \mid \text{id}$ The semantic rules are: $\text{var-list.dtype} = \text{type.dtype}$; $\text{type.dtype} = \text{integer}$ (for int); $\text{var-list}_1.\text{dtype} = \text{var-list}_2.\text{dtype}$ (propagating down). For the input "float x, y", what is the direction of information flow in the dependency graph, and why does this classify var-list.dtype as an inherited attribute?

- A) Arrows flow bottom-up from leaves to root, so it is synthesized
- B) Arrows flow top-down and sideways — from type.dtype to var-list.dtype and then further down — so it is inherited because value comes from the parent or sibling, not from the node's own children
- C) Arrows flow both ways, making it a circular attribute
- D) There are no dependency edges since dtype is a terminal attribute computed by the lexer

3. Semantic analysis is described as operating in two categories. Category 1 checks are REQUIRED by the language definition and guarantee the program can execute correctly. Category 2 checks are NOT strictly required but improve efficiency of the generated code. Which of the following correctly pairs each check to its category?

- A) Type checking = Category 2 (optimization); constant folding = Category 1 (correctness)
- B) Type checking = Category 1 (correctness); constant folding = Category 2 (optimization)
- C) Both type checking and constant folding = Category 1 (correctness)
- D) Both type checking and constant folding = Category 2 (optimization)

4. There are two methods for computing attribute values: the Parse Tree Method and the Rule-Based Method. A compiler designer must choose between them. Which statement MOST accurately describes the key practical trade-off that makes the rule-based method preferred by almost all real compilers?

- A) The parse tree method is simpler to implement but only works for S-attributed grammars; the rule-based method works for all grammar types
- B) The parse tree method builds a dependency graph once at compiler-construction time; the rule-based method rebuilds it for every input string, making it more accurate
- C) The parse tree method rebuilds the full dependency graph per input string (expensive at runtime); the rule-based method fixes a single traversal order at compiler-construction time and applies it to all inputs uniformly
- D) Both methods have identical runtime cost; the rule-based method is preferred because it produces better object code

Question 5. Static attributes and dynamic attributes differ in WHEN they are bound. A compiler writer cares most about static attributes. Given the following list — (i) the data type of a variable in C, (ii) the value of a variable at runtime, (iii) the memory layout of a static variable, (iv) the location of dynamically-allocated memory — which set contains ONLY static attributes?

- A) (i) and (iii) only
- B) (ii) and (iv) only
- C) (i), (ii), and (iii)
- D) All four are static attributes

6. In a dependency graph, edges represent computational dependencies. For the rule $\text{number}_1 \rightarrow \text{number}_2 \text{ digit}$ with equation: $\text{number}_1.\text{val} = \text{number}_2.\text{val} \times 10 + \text{digit.val}$, two edges are drawn. For the rule $\text{factor} \rightarrow (\text{exp})$ with equation: $\text{factor.val} = \text{exp.val}$, one edge is drawn. Which of the following correctly describes BOTH sets of edges?

- A) Both rules have edges flowing downward from parent to children

National University of Computer and Emerging Sciences

Islamabad Campus

B) number₁ rule: edges from number₁.val to number₂.val and digit.val (downward); factor rule: edge from exp.val to factor.val (upward)

C) number₁ rule: edges from number₂.val and digit.val to number₁.val (upward, children to parent); factor rule: edge from exp.val to factor.val (upward, child to parent)

D) number₁ rule: edges are bidirectional; factor rule: no edge is needed since factor is a leaf

7. An attribute grammar has BOTH synthesized and inherited attributes, where the synthesized attribute's value depends on an inherited attribute computed earlier (e.g., based-number grammar with 'base' inherited and 'val' synthesized). What is the correct traversal strategy for this mixed case?

A) Single postorder (bottom-up) traversal only

B) Single preorder (top-down) traversal only

C) Single combined preorder + postorder pass (visit node on the way down for inherited, on the way up for synthesized)

D) Multiple independent passes over the syntax tree

8. 'An edge from $A \rightarrow B$ means B depends on A / Compute A first, then B.' A student draws a dependency graph for the rule $\text{var-list}_1 \rightarrow \text{id}, \text{var-list}_2$ with equation $\text{var-list}_1.\text{dtype} = \text{var-list}_2.\text{dtype}$... wait — actually the equation is $\text{id}.\text{dtype} = \text{var-list}_1.\text{dtype}$ and $\text{var-list}_2.\text{dtype} = \text{var-list}_1.\text{dtype}$. What are the correct edges in this dependency graph?

A) $\text{id}.\text{dtype} \rightarrow \text{var-list}_1.\text{dtype}$ and $\text{var-list}_2.\text{dtype} \rightarrow \text{var-list}_1.\text{dtype}$

B) $\text{var-list}_1.\text{dtype} \rightarrow \text{id}.\text{dtype}$ and $\text{var-list}_1.\text{dtype} \rightarrow \text{var-list}_2.\text{dtype}$ (parent pushes value to children)

C) $\text{id}.\text{dtype} \rightarrow \text{var-list}_2.\text{dtype}$ (bypassing var-list_1 entirely)

D) No edges exist because dtype is assigned by the type terminal, not propagated

[CLO 2: Analyze and recognize the significance of the several phases through which a typical program is compiled]

Question 5:

[4.5 marks]

Consider the given CFG:

#	LHS	Production
(1)	D	$\rightarrow T \text{ VL}$
(2)	T	$\rightarrow \text{int} \mid \text{real}$
(4)	VL	$\rightarrow \text{VL}_1, \text{V} \mid \text{V}$
(6)	V	$\rightarrow \text{id} = \text{Lit} \mid \text{id}$
(8)	Lit	$\rightarrow \text{inum} \mid \text{fnum}$

Attribute Reference

The attributes used in this grammar are

T.type , VL.dtype , V.dtype , Lit.dtype , Lit.val , id.lexval , inum.lexval, fnum.lexval

Semantic Descriptions per Production

For each production, the semantic action(s) are described in plain English below. No symbolic rule is given. You must write the formal semantic rules yourself using the attribute names from Table 1.

Helper functions: **cast(value, type)** stores a numeric value under the given type.

addtype(name, type) enters an identifier into the symbol table.

National University of Computer and Emerging Sciences

Islamabad Campus

Production	What the semantic rule(s) must accomplish
(1) $D \rightarrow T \quad VL$	The type computed by T must be passed downward into the variable list VL. D itself carries no attribute.
(2) $T \rightarrow \text{int}$	T must produce a type attribute whose value is the constant integer.
(3) $T \rightarrow \text{real}$	T must produce a type attribute whose value is the constant real.
(4) $VL \rightarrow VL_1, V$	The type that VL received from above must be forwarded unchanged to both the inner variable list VL_1 and to the individual variable V.
(5) $VL \rightarrow V$	The type that VL received from above must be forwarded to the single variable V.
(6) $V \rightarrow \text{id} = \text{Lit}$	The type V received from above must be forwarded down to the literal Lit. Additionally, the identifier must be entered into the symbol table under its lexical name and declared type using <code>addtype(name, type)</code> .
(7) $V \rightarrow \text{id}$	The identifier must be entered into the symbol table under its lexical name and the type V received from above. No literal is present.
(8) $\text{Lit} \rightarrow \text{inum}$	Lit must produce a value attribute by casting the integer token's lexical value to the type Lit received from above, using <code>cast(value, type)</code> .
(9) $\text{Lit} \rightarrow \text{fnum}$	Lit must produce a value attribute by casting the real token's lexical value to the type Lit received from above, using <code>cast(value, type)</code> .

Complete the table below:

Write the complete Semantic Rules by attaching formal semantic rules to all nine productions. Where a production requires more than one rule, write each rule on a separate line in the same cell.

Production	Semantic rule(s) — write here	Marks
(1) $D \rightarrow T \quad VL$	<code>VL.inh = T.type</code>	0.5
(2) $T \rightarrow \text{int}$	<code>T.type = integer</code>	0.5
(3) $T \rightarrow \text{real}$	<code>T.type = real</code>	0.5
(4) $VL \rightarrow VL_1, V$	<code>VL₁.dtype = VL.dtype</code> <code>V.dtype = VL.dtype</code>	0.5
(5) $VL \rightarrow V$	<code>V.dtype = VL.dtype</code>	0.5
(6) $V \rightarrow \text{id} = \text{Lit}$	<code>Lit.dtype = V.dtype</code> <code>addtype(id.lexval, V.dtype)</code>	0.5
(7) $V \rightarrow \text{id}$	<code>addtype(id.lexval, V.dtype)</code>	0.5
(8) $\text{Lit} \rightarrow \text{inum}$	<code>Lit.val = cast(inum.lexval, Lit.dtype)</code>	0.5
(9) $\text{Lit} \rightarrow \text{fnum}$	<code>Lit.val = cast(fnum.lexval, Lit.dtype)</code>	0.5

[CLO 2: Analyze and recognize the significance of the several phases through which a typical program is compiled]

Question 6:

[7.5 marks]

A simple binary number language allows programs to write signed binary integer literals.

Example: **1 0 0 p** means +4 in decimal ($1 \times 4 + 0 \times 2 + 0 \times 1 = 4$, positive).

National University of Computer and Emerging Sciences

Islamabad Campus

Example: **1 1 0 n** means -6 in decimal ($1 \times 4 + 1 \times 2 + 0 \times 1 = 6$, negative)

Given Grammar & Semantic Rules

Grammar Rule	Semantic Rules
$signed_num \rightarrow bits\ signchar$	$signed_num.val =$ $if\ signchar.sign = neg$ $then\ -\ bits.mag$ $else\ bits.mag$ $bits.wt = 1$
$signchar \rightarrow p$	$signchar.sign = pos$
$signchar \rightarrow n$	$signchar.sign = neg$
$bits_1 \rightarrow bits_2\ bit$	$bits_1.mag =$ $bits_2.mag + bit.val \times bit.wt$ $bits_2.wt = bits_1.wt \times 2$ $bit.wt = bits_1.wt$
$bits \rightarrow bit$	$bits.mag = bit.val \times bit.wt$ $bit.wt = bits.wt$
$bit \rightarrow 0$	$bit.val = 0$
$bit \rightarrow 1$	$bit.val = 1$

Draw Dependency graph on this parse tree – write attributes, draw edges, perform calculations for attributes wherever needed and write final answer. Blocks represent the attributes attached to each non-terminal.

